# "Two-Stagification": Job Dispatching in Large-Scale Clusters via a Two-Stage Architecture

Mert Yildiz*, Alexey Rolich*, Andrea Baiocchi*
*Dept. of Information Engineering, Electronics, and Telecommunications (DIET), University of Rome Sapienza, Italy
{mert.yildiz,alexey.rolich,andrea.baiocchi}@uniroma1.it

2025 23rd Mediterranean Communication and Computer Networking Conference (MedComNet) | 979-8-3315-0327-7/25/$31.00 ©2025 IEEE | DOI: 10.1109/MEDCOMNET65822.2025.11103543

***Abstract*—A continuing effort is devoted to devising effective dispatching policies for clusters of First Come First Served servers. Although the optimal solution for dispatchers aware of both job size and server state remains elusive, lower bounds and strong heuristics are known. In this paper, we introduce a two-stage cluster architecture that applies classical Round Robin, Join Idle Queue, and Least Work Left dispatching schemes, coupled with an optimized service-time threshold to separate large jobs from shorter ones. Using both synthetic (Weibull) workloads and real Google data center traces, we demonstrate that our two-stage approach greatly improves upon the corresponding single-stage policies and closely approaches the performance of advanced size- and state-aware methods. Our results highlight that careful architectural design—rather than increased complexity at the dispatcher—can yield significantly better mean response times in large-scale computing environments.**

***Index Terms*—Data centers; Dispatching; Scheduling; Multiple parallel servers; Real-world workload; Large-Scale multi-server system; Workload traffic measurements.**

## I. Introduction

Finding effective and easily implementable algorithms for distributing the workload in server clusters is among the top issues in the framework of cloud and edge computing. Given a set of servers to which a sequence of jobs is offered, possibly broken down into several tasks, dispatching policies assign each job or task to one server. Dispatching is therefore a workload distribution function, unlike scheduling, which is implemented *inside* each server and defines when and how much of the processing facility of the server is assigned to each job or task dispatched to that server.

Dispatching decisions are typically assumed to be immediate, i.e., upon job arrival. Dispatching policies can be classified according to the amount of information available at the dispatching process. The simplest policies are stateless and non-anticipative, i.e., no information is available on the current state of servers, nor are job/task sizes known at time of dispatching. In this framework, we distinguish between memoryless strategies (e.g., random selection of a server), or strategies that keep a memory of past choices (e.g., Round Robin (RR)). Stateful dispatching policies base their decisions on knowledge of the current system state, e.g., by means of server polling. Join Idle Queue (JIQ) and Join Shortest Queue (JSQ) are examples of this class of policies. Finally, anticipative policies (also known as size-aware policies) exploit knowledge of job/task size upon arrival, before the job/task is run. Least Work Left (LWL) and Size Interval Task Assignment (SITA) are classic examples of this kind of policies. Other classification criteria are possible, e.g., based on time variability of policy parameters. Static policies maintain values of their parameters unchanged over time, while dynamic policies adapt their parameters and rules, e.g., based on running time estimates of relevant quantities.

While many dispatching and scheduling policies have been defined and analyzed under different assumptions (e.g., see [1]–[4]), still many open issues are on the table [5]. We focus on a server cluster made up of $n$ identical servers that serve jobs according to First Come First Served (FCFS) scheduling. A full understanding of even such a simple multi-server cluster system is yet to be achieved. The optimal dispatching policy is known in some cases, e.g., round robin is optimal under the assumption that nothing is known to the dispatching process, except for the history of previous dispatching decisions; SITA is optimal when job sizes are known upon arrival, but no server state information is available. However, to date, the exact optimal policy in the general case, where job size is known upon arrival as well as the current status of servers, is not characterized, even in the relatively simple M/G-type model setting (i.e., Poisson arrival of jobs, renewal job sizes).

On the other hand, several powerful heuristics have been proposed. Dynamic sequential policies are defined in [6]. The key idea is to start from a baseline policy, to define a sequential ordering of servers. Servers are then polled according to the considered sequential order, checking a condition, that can depend on server status and on job size. The job is assigned to the first server that matches the condition. The optimal policy is derived numerically in [7] from Bellman-type optimality equations in a cluster of servers with Poisson job arrivals, then generalized to renewal inter-arrival times. This approach however calls for significant complexity with growing server cluster size. A general lower bound of Mean Response Time (MRT) for the class of stateful, anticipative dispatching policies is established in [8]. A heavy-traffic delay optimal heuristic policy is defined as well, named Controlled Asymmetry Reduces Delay (CARD) [8]. A version of this algorithm, referred to as multi-band flexible CARD, is defined for arbitrary server cluster size. This policy is shown to yield very good performance at every load level, not just asymptotically in heavy traffic.

A common trait of proposed heuristic policies is their complexity. They require knowledge of job size upon arrival, and knowledge of current server status, which practically requires message exchange between the dispatching process and servers. An effort towards simpler yet effective dispatching policies was the motivation of [2], where Join Below Threshold (JBT) is defined and proved to be heavy-traffic delay optimal. This policy is a generalization of the sub-optimal JIQ policy. The dispatcher keeps track of servers whose backlog is less than $r$ jobs (the threshold) and jobs are assigned to one of those servers, if available, randomly to any server otherwise. The dynamic version of this policy adapts the threshold according to an estimate of the current backlog of servers. The estimate is obtained by means of polling of a few randomly selected servers.

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on October 27,2026 at 15:10:51 UTC from IEEE Xplore. Restrictions apply.

The core problem addressed in this work is designing effective, implementable dispatching policies for FCFS server clusters. While advanced state- and size-aware strategies exist, the optimal policy remains unknown and existing heuristics tend to be complex. The motivation is to assess whether simple, low-overhead policies—requiring minimal system knowledge—can match the performance of more sophisticated methods in large-scale cloud and edge environments.

In this paper, we explore how to achieve comparably good performance in terms of MRT by means of very simple dispatching policies. Our key contributions are:

- Demonstrating that simple, size- and state-agnostic policies (e.g., RR) can match the performance of more complex strategies.
- Showing that a two-stage server architecture significantly improves performance over traditional single-stage dispatching, benefiting not only RR but also JIQ and LWL.

In the two-stage system, jobs are simply offered to the first stage of servers upon their arrival. If a job is not completed within a given time threshold, it is stopped, moved to the next stage, and restarted. We show that it is possible to improve over size-aware best heuristics, provided the threshold and the sizes of the two stages are properly optimized. While our current analysis does not delve into the overhead potentially introduced by the job transfer between stages, we acknowledge its relevance. This aspect will be addressed in future work, where we plan to quantify its impact on overall system performance.

To that end, we first consider an artificially generated workload, based on Poisson arrivals and renewal job sizes, according to the classic M/G-type model setting. To confirm obtained results in a more realistic environment, we exploit high-quality workload measurements of production data centers, specifically those published recently by Google [9]–[14]. In that dataset, job arrivals do not follow a Poisson process, and the job/task sizes do not adhere to a negative exponential distribution [12]. On the contrary, the distribution of service times is highly variable and exhibits an extremely heavy-tailed behavior, whereby a small fraction of the very largest jobs comprises most of the load. In prior empirical studies of computational workloads [15]–[19], the authors found that the top 1% of jobs comprise 50% of the load. The heavy-tailed property exhibited in Google data centers is even more extreme, namely, the largest (most compute-intensive) 1% of jobs comprise about 99% of the load [10]. This makes performance evaluation based on Google's measured workload particularly challenging for dispatching policies.

The rest of the paper is organized as follows. Section II provides a brief explanation of the considered system model and workload characteristics, including the main features of workload measurement data. A concise description of considered dispatching policies is also given. Numerical results of data-driven simulations for both artificial and experimental workloads are presented in Section III. Finally, conclusions are drawn in Section IV.

## II. System Model and Dispatching Policies

We consider $n$ identical FCFS servers having processing speed $\mu$. Jobs are offered to this server cluster with mean arrival rate $\lambda$ and size distributed according to a continuous random variable $S$ with Complementary Cumulative Distribution Function (CCDF) $G(x) = \mathcal{P}(S > x)$. Service time, i.e., time required to complete a job on a server, is given by $X = S/\mu$. The scale parameter $\mu$ is adjusted to impose that the coefficient of utilization $\rho$ averaged across servers has a desired value, namely, it is

$$\rho = \frac{\lambda \mathrm{E}[S]}{n\mu} \tag{1}$$

We also denote with $C_S$ the Coefficient Of Variation (COV) of service times, defined as the ratio of the standard deviation of service time to the mean service time. Jobs are instantaneously dispatched to one of the servers upon their arrival. When dealing with experimental workload data, we account for the fact that a job can possibly be composed of several independent tasks. In that case, tasks get assigned individually to possibly different servers, according to the considered dispatching policy. In any case, the reference metric is mean *job* response time. The job response time $R$ is defined as the time elapsing since job arrival until the last task of the job gets completed. The MRT is displayed either against $\rho$ or against the server cluster size $n$. In either case, the overall computational power of the cluster, $n\mu$, is kept constant as $n$ varies, by adjusting $\mu$. This makes the scaling of performance as $n$ varies non-trivial, given that the overall amount of computational resource is the same for each value of $n$, but the parallelism degree of the server cluster varies.

### A. Artificial workload

To gather extensive performance evaluation data, we consider an M/G-type theoretical model setting. We assume Poisson arrival of jobs, each one composed of a single task. Job size is assumed to be independently generated for each job, drawn from the CCDF $G(x)$. In our numerical evaluation, we consider a Weibull distribution, i.e.,

$$G(x) = e^{-\left(\frac{x}{a}\right)^b}, x > 0. \tag{2}$$

The parameters $a$ and $b$ are found from the first two moments of job size, taking into account that

$$\mathrm{E}[S] = a\Gamma\left(1 + \frac{1}{b}\right) = \frac{a}{b}\Gamma\left(\frac{1}{b}\right) \tag{3}$$

$$\mathrm{E}[S^2] = \frac{2a^2}{b}\Gamma\left(\frac{2}{b}\right) \tag{4}$$

where $\Gamma(z) = \int_0^\infty u^{z-1}e^{-u}\,du$ is the Euler Gamma function.

Specifically, we set $\mathrm{E}[S] = 1$ (so that the mean job service time on a standard server of speed $\mu = 1$ is the unit of measure of time), and $\mathrm{E}[S^2] = (\mathrm{E}[S])^2\left(1 + \mathrm{COV}^2\right)$.

The mean arrival rate is fixed to achieve a desired load $\rho = \lambda \mathrm{E}[S]$, while the processing speed $\mu$ is set to $\mu = 1/n$ so that the overall computational budget of a server cluster made up of $n$ servers be fixed at 1.

### B. Experimental workload dataset

This section describes the Google traffic dataset released in 2020, capturing one month of user and developer activity in May 2019. It provides detailed resource usage data, such as Central Processing Unit (CPU) and Random Access Memory (RAM), across eight Google data centers ("Borg cells").

Traffic data includes five tables detailing users' resource requests, machine processing, and task evolution within Borg's scheduler. Users submit jobs comprising one or more tasks (instances), each requiring specific CPU time and memory

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on October 27,2026 at 15:10:51 UTC from IEEE Xplore. Restrictions apply.

space. Resource units are based on Google Normalized Computing Unit (GNCU) (default machine computational power). Exposed CPU time associated with the execution of a task assumes that servers are equipped with one GNCU.

Jobs are composed of one or more tasks, each of which follows a life cycle. Tasks are queued or processed based on load and classified upon completion as "FINISH" (success) or "KILL," "LOST," or "FAIL" (failure) [9]. Only "FINISH" tasks were analyzed, as they provide reliable resource data. The dataset includes task identifiers, CPU time (min, max, avg over 300-sec windows), and 1-sec samples per window of resource usage.

Digging into Google's data, the workloads have been reconstructed and the following key features are used in this work[1].

- $J_j$ - number of tasks belonging to job $j$.
- $A_j$ - Arrival time (in seconds) of job $j$. All tasks belonging to the same job arrive at this time.
- $S_{ij}$ - CPU time (in seconds) required to process task $i$ of job $j$ on a reference server equipped with one GNCU.

The whole considered dataset (31 days worth of measured activity in Borg cell c) consists of 4399670 jobs, comprising 7010742 tasks. Most jobs consist of a single task (96.6%), yet "monster" jobs are also recorded, with the largest ones comprising up to 11160 tasks. The mean job CPU time is 10.83 s [2]. Only 13.25% of jobs require CPU time larger than the mean, while 99% of jobs have CPU time less than 32.26 s, 99.9% of jobs less than 452.95 s, and 99.99% of jobs less than 4974.2 s. The largest 0.1% of jobs is responsible for 67.6% of the overall required CPU effort. These few numbers give evidence of the extreme variability of workload described in Google's data.

The mean arrival rate of jobs $\lambda$ is estimated from data, as well as the job and task size distribution (used to set thresholds in some dispatching policies). The server speed $\mu$ is adjusted for each given cardinality $n$ of the server cluster to achieve a desired server utilization coefficient $\rho$, according to Equation (1).

### C. Dispatching policy

For the ease of the reader, we provide a concise summary of the considered dispatching policies.

*1) Round Robin:* This policy does not rely on knowledge of either job size or server state, yet it exploits the history of decisions on job assignment. The $k$-th arriving job is assigned to server $1 + (k-1) \mod n$, for $k \geq 1$.

*2) Join Idle Queue:* This is an example of a policy that relies only on some server state information but does not require any information about job size. The dispatching process maintains a look-up table of $n$ bits. All bits are set to 1 initially. Upon job arrival, the dispatcher picks a server at random among those having their bit at 1, sends the job to that server, and resets the corresponding bit in the table. If no bit is at 1 in the table, the dispatcher sends the job to a server picked uniformly at random. Whenever a server becomes idle (i.e., the last job with that server leaves the server, having been completed), the server sends a message to the dispatching process, so that the dispatching process can restore the server's bit at 1 in its table.

*3) Least Work Left:* This policy requires full knowledge of job size upon arrival, as well as keeping track of the current status of servers. Let $W_j(t)$ denote the unfinished work at server $j$ at time $t$. If servers are work-conserving, the dispatching process can keep track of $\mathbf{W}(t) = [W_1(t), \ldots, W_n(t)]$ by knowing job sizes and the dispatching decision history. Upon arrival of a job at time $a$, the job gets assigned to the server having minimum unfinished work, i.e., to server $j^* = \arg\min[W_1(a), \ldots, W_n(a)]$. Ties are broken at random.

*4) Multi-band flexible CARD:* This is an example of dynamic policy exploiting both server status and job size knowledge. Let thresholds $m_i$ and $c_i$ be defined as follows. The thresholds $m_i$, $i = 1, \ldots, n$, are defined so that the fraction of system load due to jobs with size falling in the interval $[m_i, m_{i+1})$ be equal to $1/n$ for $i = 1, 2, \ldots, n-1$. Moreover, we set $m_0 = 0$ and $m_{n+1} = \infty$. Then, the fraction of system load due to jobs whose size is less than $m_1$ or larger than $m_n$ must be equal to $1/(2n)$. It is easy to verify that this is equivalent to setting $m_i$ so that the following equality holds

$$\frac{1}{\mathrm{E}[S]} \int_0^{m_i} x f(x)\, dx = \frac{i - 1/2}{n} \tag{5}$$

for $i = 1, \ldots, n$, where $f(x)$ is the Probability Density Function (PDF) of job sizes. As for the thresholds $c_i$, Authors of [8] suggest to set $c_i = m_i/\sqrt{1-\rho}$, for $i = 1, \ldots, n-1$.

Then, the CARD policy works as follows. Upon job arrival at time $a$, unfinished work levels $W_i(a)$, $i = 1, \ldots, n$, at servers are sorted in ascending order. Let $W_{(i)}(a)$ denote the ordered sequence of unfinished work levels, i.e., $W_{(1)}(a) \leq W_{(2)}(a) \leq \cdots \leq W_{(n)}(a)$. Let also $K_{(i)}(a)$ denote the index of the $i$-th ranked server, i.e., the server having the $i$-th smallest unfinished work amount. Let $s$ denote the size of the newly arrived job.

1) If $s < m_1$, the job gets assigned to server $K_{(1)}(a)$.
2) if $s > m_n$, the job gets assigned to server $K_{(n)}(a)$.
3) If $s \in [m_i, m_{i+1})$, the job gets assigned to server $K_{(i)}(a)$ if $W_{(i)}(a) \leq c_i$, to server $K_{(i+1)}(a)$ otherwise $(i = 1, \ldots, n-1)$.

*5) Two-stage dispatching:* The two-stage dispatching architecture was first introduced in [13], further refined in [14], and in this paper, we generalize and apply its most effective variant across different dispatching policies. The mechanism operates as follows: the servers are divided into two sub-clusters (or stages) of sizes $n_1$ and $n_2 = n - n_1$. Upon arrival, a job is dispatched to one of the $n_1$ servers in the first stage according to a chosen dispatching policy $D$ (e.g., RR, JIQ, or LWL). If the job is not complete within a time interval of $\theta/\mu$ (i.e., if its size exceeds $\theta$), it is preempted and transferred to the second stage, where it is assigned to one of the $n_2$ servers using the same policy $D$ and then restarted to be completed in the second stage. All servers operate under FCFS scheduling. The only additional mechanism is a timer on the first-stage servers; if the timer expires, the job currently running is halted and moved to the second stage.

Notably, when using RR, this policy requires neither knowledge of the job size nor monitoring of server status during dispatching. In the case of JIQ, only the idle/busy state of each server is needed, a requirement that is straightforward to implement in realistic systems. Its performance obviously

[1]The dataset for all eight data centers is available at .https://github.com/MertYILDIZ19/Google_cluster_usage_traces_v3_all_cells

[2]We remind that all CPU times are given with reference to a standard server equipped with 1 GNCU.

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on October 27,2026 at 15:10:51 UTC from IEEE Xplore. Restrictions apply.

depends on the two meta-parameters $n_1$ and $\theta$. In this work we consider a static optimization of those parameters, leaving for further work the definition of an algorithm for automatic adaptation of those parameters.

## III. Numerical Results

In this section, we evaluate the performance of various dispatching policies through simulations. Our objective is to address the following key questions:

- How does the degree of parallelism (i.e., the number of servers) affect performance, and what is the optimal server range for performance optimization?
- How does the Two-Stage dispatching policy compare to the traditional Single-Stage dispatching and the size and state-aware, heavy-traffic optimal policy, CARD?
- How do the Two-Stage and CARD policies perform under Poisson arrivals and Weibull-distributed service times compared to real-world Google workload traces?

As discussed in the preceding sections, we employ four benchmark dispatching policies: RR, JIQ, LWL, and CARD. Although dispatching policies have been extensively studied, we selected these due to their varying levels of sophistication. Specifically, RR serves as a simple, stateless baseline; JIQ is a stateful, but size-agnostic policy; and LWL is the most sophisticated of the three, incorporating both size and state information. We also include CARD in our evaluation because it is provably optimal [8] in heavy-traffic regimes for size- and state-aware dispatching. The point we aim to make is that a significant performance gain can be reaped by changing the server cluster architecture from single-stage to two-stage. Therefore, we apply RR, JIQ, and LWL dispatching in a two-stage framework and evaluate their performance accordingly.

Heavy-tailed distributions are prevalent in computer systems and networks (e.g., [20]), and their tendency to result in high mean response times underscores the importance of an effective dispatching policy. To reproduce this variability we consider artificially generated workloads in Section III-A and real-world workloads obtained from Google measurements in Section III-B.

### A. Performance under Weibull-Distributed Workload

In this Section, we first evaluate the system under an M/G-type model in which job sizes follow a Weibull distribution with a mean normalized to 1. To capture both moderate- and heavy-variability regimes, we set the COV to either 1 or 10. We compare the mean response time under different dispatching policies by normalizing it with respect to the theoretical mean response time of an equivalent single-server FCFS M/G/1 queue, $E[R_{M/G/1}]$. Our goal is to examine how these policies perform as we vary both the number $n$ of servers and the system load $\rho$, while keeping total processing capacity fixed.

First, the effect of the server cluster size is evaluated. We will show that as $n$ grows, near-optimal performance can be achieved even with relatively simple policies, while for low or moderate values of $n$ smart dispatching policies have their merit. Figure 1 and Figure 2 plot the normalized mean response time as a function of the number of servers for COV equal to 1 and 10, respectively, for the three basic dispatching policies RR, JIQ, and LWL, with $\rho = 0.8$. The lower bound derived in [8] is shown as well.

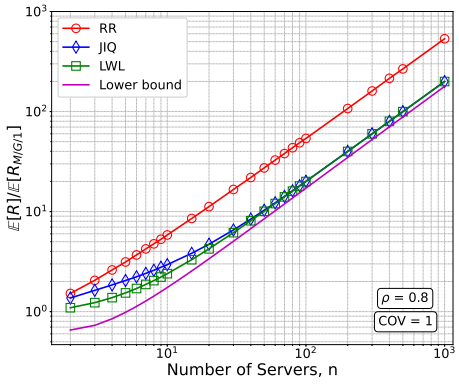



Figure 1. MRT as a function of the number of servers. **Note:** COV = 1.

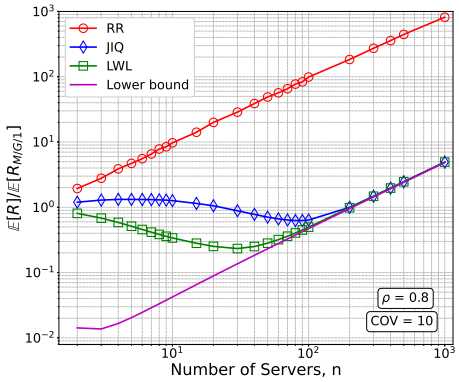



Figure 2. MRT as a function of the number of servers. **Note:** COV = 10.

These results offer some key insights. When $n$ becomes large enough, both JIQ and LWL perform almost as well as the theoretical lower bound. On the contrary, RR exhibits an offset with respect to the lower bound, the bigger the higher the COV. Hence, *for large parallelism degree of the cluster server, there is no need to resort to size-aware dispatching policies*. Basic state information (namely, idle/busy server state) is enough to reap near-optimal mean response time. This is not a trivial result, given that the overall computational power of the server cluster is fixed to constant values as $n$ varies, by scaling the server speed $\mu$. Therefore, service time grows as $n$ grows.

*A significant offset with respect to optimal performance arises at moderate levels of* $n$. For $n \in \{2, \ldots, 100\}$, JIQ and LWL consistently outperform RR for both $\mathrm{COV} = 1$ (negative exponential distribution) and $\mathrm{COV} = 10$. In the exponential regime ($\mathrm{COV} = 1$), JIQ and LWL yield almost the same mean response times, indicating that knowing job sizes (LWL) offers a minimal advantage over purely state-based dispatching (JIQ). By contrast, under the heavy-tailed regime ($\mathrm{COV} = 10$), the impact of large jobs is magnified, making any policy that leverages state information substantially more effective than the baseline RR. From these observations, we conclude that the most pronounced performance distinctions emerge in moderate-scale systems, between a few to a few hundred servers. Beyond a sufficiently high degree of parallelism, both JIQ and LWL closely match the lower bound, rendering the choice of dispatching policy less critical. Consequently, evaluation and optimization will focus on the range $2 \leq n \leq 100$, where differences between dispatching strategies are most apparent. Within this critical region, we compare the performance of both single-stage traditional and CARD with two-stage dispatching policies.

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on October 27,2026 at 15:10:51 UTC from IEEE Xplore. Restrictions apply.

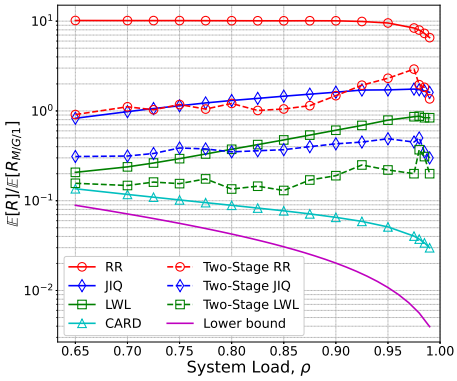



Figure 3. MRT as a function of the system load. **Note:** COV = 10 and n = 10.

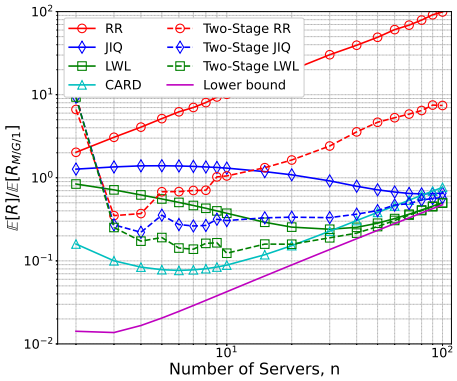



Figure 4. MRT as a function of the number of servers. **Note:** COV = 10 and $\rho = 0.8$.

Before proceeding, we clarify the two-stage setting as follows.

**Two-stage Architecture and Threshold Selection:** For the two-stage variants of RR, JIQ, and LWL, we optimize two key parameters: (i) the threshold $\theta$, selected from a set of quantiles of the job size distribution, and (ii) the first-stage size $n_1$, with the second stage comprising $n_2 = n - n_1$ servers. By testing various quantile-based thresholds and different splits of $n_1$ and $n_2$, we identify an effective boundary for offloading jobs from the first to the second stage. In practice, this approach significantly enhances performance for shorter jobs, as it isolates heavy jobs onto the second stage while allowing shorter jobs to be completed rapidly in the first stage.

Figure 3 shows the normalized mean response time, $\mathbb{E}[R]/\mathbb{E}[R_{M/G/1}]$, as a function of the system load $\rho$ setting the number of server n = 10. One can see that each policy's performance varies smoothly with $\rho$, with RR (red) generally yielding the highest mean response times at moderate to high loads, and CARD (cyan) consistently remaining the best-performing single-stage policy across the entire load range. Figure 3 illustrates that the optimized two-stage RR variant (dashed red) significantly narrows the gap between single-stage RR and more sophisticated policies, highlighting how selectively separating large jobs can substantially improve overall performance. Nevertheless, at extremely high loads close to $\rho = 1$, the advantage of knowing job sizes (LWL) or at least the state of the server (JIQ) still emerges in slightly lower response times.

In Figure 4, we fix $\rho = 0.8$ and vary the number of servers $n$ to clearly illustrate the impact of server parallelism. When $n$ is small, the two-stage policies perform poorly; however, increasing the server count to around 10–20 dramatically reduces the mean response time. In realistic scenarios, where one does not employ an extremely small number of servers (which would require each server to be exceptionally powerful to maintain a constant total capacity), the RR policy, in particular, fails to achieve satisfactory performance. LWL and JIQ, by contrast, already achieve good performance for small $n$ and then see diminishing returns beyond moderate-scale configurations, especially as the cluster size grows further. Eventually, when $n$ is very large, the lines tend to converge, indicating that high parallelism weakens the distinction among policies. In all configurations, CARD continues to demonstrate the best single-stage performance, while the threshold-optimized two-stage algorithms remain close behind, underscoring the effectiveness of migrating heavier jobs to a second stage to shield latency-sensitive, shorter jobs from undue delays.

### *B. Performance under Real Google Workload*

We now turn to a real-world Google data center workload that encompasses a full day of measured user activity. Unlike the artificially generated workload, jobs in this dataset might comprise multiple tasks, each with its own CPU requirement (i.e., task size). An in-depth analysis of Google workload data shows that [12]: (i) all tasks belonging to the same job arrive at the same time (within the accuracy of timestamps); (ii) tasks belonging to the same job appear to be independent, i.e., there is no evidence of the need for completing a given task to start running another task of the same job. Hence, in our simulations, tasks are dispatched individually according to the selected policy, i.e., RR, JIQ, LWL, CARD, and their two-stage variants. Mean response time is measured as *job* completion time, i.e., the elapsed time since the job's arrival, until *all* of its tasks have finished.

Figure 5 and Figure 6 plot the mean job response time (in hours) as we vary the system load $\rho$ for $n = 10$, and the cluster size $n$ for $\rho = 0.8$, respectively.

Focusing first on Figure 5, it is apparent that exploiting state or size information makes little difference among RR, JIQ, and LWL, while changing the architecture of the server cluster from one to two-stage brings about a major improvement of mean response time performance. CARD (cyan curves) consistently outperforms the other policies, thus showing that state and size information *can* make a difference if played in the right way. It is however remarkable that a good part of the performance advantage gained by CARD over single-stage policies can be reaped by adopting the baseline RR policy but moving to two-stage architecture.

Turning to Figure 6 we observe that CARD still boasts a noticeable performance edge, but two-stage JIQ and LWL (dashed blue and green curves) remain close to CARD and clearly outperform their single-stage counterparts. This shows that carefully splitting the cluster can substantially improve JIQ and LWL, especially under moderate to large values of $n$. It is also notable that the two-stage LWL policy offers no performance advantage over the simpler two-stage JIQ. In the single-stage setting, JIQ even outperforms LWL in some cases, highlighting that policies leveraging more information—such as job size—do not necessarily yield better performance than simpler, state-based alternatives.

In this case, LWL fails to provide some imbalance among different servers, as achieved naturally by JIQ due to randomization of task assignments when all servers are busy.

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on October 27,2026 at 15:10:51 UTC from IEEE Xplore. Restrictions apply.

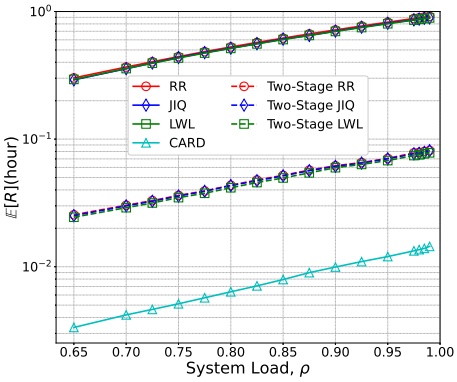



Figure 5. MRT as a function of the system load. Google data has been used including the task-level data structure. **Note:** $n = 10$

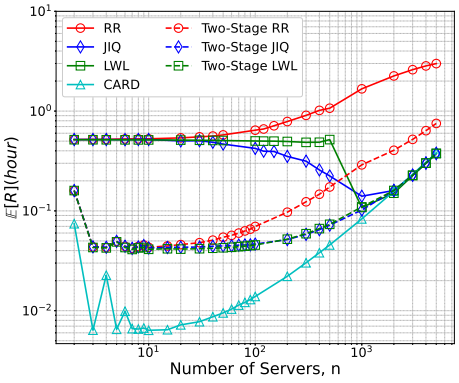



Figure 6. MRT as a function of the number of servers. Google data has been used including the task-level data structure. **Note:** $\rho = 0.8$

Overall, these results confirm that real data center workloads are heavily skewed in both job sizes and concurrency levels. While CARD leverages both queue state and job-size knowledge to achieve robust performance, the two-stage approach continues to offer significant improvements for RR and yields competitive outcomes for JIQ and LWL. This indicates that architectural modifications can be a powerful alternative to deploying more complex, information-intensive dispatching algorithms when tackling real, large-scale workloads.

## IV. Conclusions

In this paper, we demonstrated that a two-stage architecture can closely approach the performance of advanced size- and state-aware policies in FCFS server clusters. By introducing a service-time threshold and partitioning servers into two stages, even simple policies like RR exhibit substantial gains—especially under heavy-tailed workloads, where isolating large jobs shields short ones from excessive delays.

Experiments on synthetic Weibull workloads and real Google traces confirm that CARD, with full size and state-awareness, remains the top performer. However, two-stage variants of JIQ and LWL often achieve comparable MRT with reduced implementation complexity and limited coordination, offering a practical alternative for large-scale systems with high job size variability.

For future work, we aim to design an adaptive mechanism that continuously learns the optimal threshold by tracking service times of arriving jobs. This would enable the system to respond dynamically to workload shifts and better isolate large jobs. We also plan to investigate a two-stage version of CARD, to determine whether it can surpass its single-stage performance. Finally, we intend to quantify the overhead of transferring jobs between stages, which was not addressed in this study but may impact performance under high load or resource contention.

## Acknowledgment

This work was supported by the European Union - Next Generation EU under the Italian National Recovery and Resilience Plan (NRRP), Mission 4, Component 2, Investment 1.3, CUP B53C22004050001, partnership on “Telecommunications of the Future” (PE00000001 - program “RESTART”)

## References

[1] M. Harchol-Balter, *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*. Cambridge University Press, 2013.

[2] X. Zhou, F. Wu, J. Tan, Y. Sun, and N. Shroff, “Designing Low-Complexity Heavy-Traffic Delay-Optimal Load Balancing Schemes: Theory to Algorithms,” *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 1, no. 2, Dec. 2017.

[3] M. Kumar, S. Sharma, A. Goel, and S. Singh, “A comprehensive survey for scheduling techniques in cloud computing,” *Journal of Network and Computer Applications*, vol. 143, pp. 1–33, 2019.

[4] S. Tang, Y. Yu, H. Wang, G. Wang, W. Chen, Z. Xu, S. Guo, and W. Gao, “A Survey on Scheduling Techniques in Computing and Network Convergence,” *IEEE Communications Surveys & Tutorials*, vol. 26, no. 1, pp. 160–195, 2024.

[5] M. Harchol-Balter, “Open problems in queueing theory inspired by datacenter computing,” *Queueing Syst. Theory Appl.*, vol. 97, pp. 3–37, feb 2021.

[6] E. Hyytiä and R. Righter, “On Sequential Dispatching Policies,” in *2022 32nd International Telecommunication Networks and Applications Conference (ITNAC)*, 2022, pp. 1–6.

[7] ——, “Towards the optimal dynamic size-aware dispatching,” *Performance Evaluation*, vol. 164, p. 102396, 2024.

[8] R. Xie, I. Grosof, and Z. Scully, “Heavy-Traffic Optimal Size- and State-Aware Dispatching,” *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 8, no. 1, Feb. 2024.

[9] J. Wilkes, “cluster-data/Clusterdata2019,” https://github.com/google/cluster-data/tree/master, 2019.

[10] M. Tirmazi, A. Barker, N. Deng, M. E. Haque, Z. G. Qin, S. Hand, M. Harchol-Balter, and J. Wilkes, “Borg: the next generation,” in *Proceedings of the Fifteenth European Conference on Computer Systems*. Association for Computing Machinery, 2020.

[11] Y. Sfakianakis, E. Kanellou, M. Marazakis, and A. Bilas, “Trace-Based Workload Generation and Execution,” in *Euro-Par 2021: Parallel Processing: 27th International Conference on Parallel and Distributed Computing, Lisbon, Portugal, September 1–3, 2021, Proceedings*. Springer-Verlag, 2021, p. 37–54.

[12] M. Yildiz and A. Baiocchi, “Data-Driven Workload Generation Based on Google Data Center Measurements,” in *2024 IEEE 25th International Conference on High Performance Switching and Routing (HPSR)*, 2024, pp. 143–148.

[13] M. Yildiz, A. Rolich, and A. Baiocchi, “The Merit of Simple Policies: Buying Performance With Parallelism and System Architecture,” 2025. [Online]. Available: https://arxiv.org/abs/2503.16166

[14] ——, “Dispatching Odyssey: Exploring Performance in Computing Clusters under Real-world Workloads,” 2025. [Online]. Available: https://arxiv.org/abs/2504.10184

[15] M. Crovella, M. Harchol-Balter, and C. Murta, “Task assignment in a distributed system: Improving performance by unbalancing load,” in *Proceedings of the ACM SIGMETRICS Joint International Conference on Measurement and Modeling of Computer Systems*, 1998, pp. 268–269.

[16] M. Harchol-Balter, “Network analysis without exponentiality assumptions,” Ph.D. dissertation, University of California at Berkeley, 1996.

[17] M. Harchol-Balter and A. Downey, “Exploiting process lifetime distributions for dynamic load balancing,” in *Proceedings of ACM SIGMETRICS*, 1996, pp. 13–24.

[18] M. Harchol-Balter, “The effect of heavy-tailed job size distributions on computer system design,” in *Proceedings of ASA-IMS Conference on Applications of Heavy Tailed Distributions in Economics, Engineering and Statistics*, 1999.

[19] M. Harchol-Balter, B. Schroeder, N. Bansal, and M. Agrawal, “Size-based scheduling to improve web performance,” *ACM Transactions on Computer Systems (TOCS)*, vol. 21, no. 2, pp. 207–233, 2003.

[20] A. Mahanti, N. Carlsson, A. Mahanti, M. Arlitt, and C. Williamson, “A Tale of the Tails: Power-Laws in Internet Measurements,” *IEEE Network*, vol. 27, no. 1, pp. 59–64, 2013.

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on October 27,2026 at 15:10:51 UTC from IEEE Xplore. Restrictions apply.